Herald

ATMOSphere ⍰ Herald — Daemon Deployment Runbook (Operator Guide)

Field	Value
Created	2026-05-30
Status	active
Scope	Host-side deployment of the two ATMOSphere⍰Herald daemons: `pherald watch` (outbound SSoT→notify) and `pherald listen` (inbound Telegram→CRUD).
Work-items	HRD-152 (inbound actions + ItemMutator), HRD-153 (`pherald watch` daemon), HRD-156 (live MTProto QA — PENDING), HRD-157 (host deployment).
Tooling	`deploy/atmosphere-herald/` — `install.sh`, `uninstall.sh`, `seed-subscribers.sh`, plus the systemd + launchd unit templates.

> **Honesty notice (§107 anti-bluff).** This runbook + the `deploy/atmosphere-herald/` tooling are **install tooling and a procedure** — they do **NOT** by themselves prove the daemons run live on the nezha host. The live proof is the operator running `install.sh` on the host and completing the **§Verify** step (a real SSoT mutation reaching Telegram + a real inbound command mutating a row). No live-evidence claim is made here that has not been captured under `docs/qa/<run-id>/`. The installer was exercised in `--dry-run` against a fixture Herald home (validation, path substitution, and fail-loud paths confirmed); that is build-tooling verification, not a live daemon run.

Table of contents

---

§1. What gets deployed

Two long-running daemons, one per direction of the ATMOSphere↔Herald integration:

Daemon	Command	Direction	What it does
watch	`pherald watch --db <db> --poll 1s`	SSoT → Telegram	Opens the shared workable-items SQLite SSoT, watches it (+ the `Issues.md`/`Fixed.md` trackers + WAL sidecars), diffs every change, and fans each per-property delta out to the configured channel(s).
listen	`pherald listen --db <db> --docs-dir docs --qa-out-dir …`	Telegram → CRUD	Runs the Telegram `getUpdates` long-poll, dispatches each operator message to Claude Code, parses the `<<<HERALD-REPLY>>>` action, and applies `item.update` / `item.delete` / `investigation.start` + `CONFIRM` mutations to the SSoT via `--db`.

On Linux (the nezha host) these run as **systemd user services**; on macOS they run as **launchd user agents**. `install.sh` picks the right one automatically (`uname -s`). Both are configured for restart-on-failure and log to the journal (systemd) or to `qa-results/*.log` (launchd).

The unit files are **templates** under `deploy/atmosphere-herald/{systemd,launchd}/`; `install.sh` substitutes the resolved paths (`@HERALD_HOME@`, `@PHERALD_BIN@`, `@ENV_FILE@`, `@WORKABLE_DB@`, `@DOCS_DIR@`, `@QA_OUT_DIR@`) into the installed copy. Do not hand-edit the installed copy — re-run `install.sh`.

Routing model (read this — HRD-156 caveat)

`pherald watch` does **not** yet resolve recipients from the Postgres `subscribers` table. It fans every change out to the **per-channel configured** `Target` — i.e. `HERALD_TGRAM_CHAT_ID`, the ATMOSphere main group's chat id (see `WORKABLE_ITEMS_INTEGRATION.md` §4.4 and `watch.go::buildWatchNotifier`). PG-backed per-subscriber resolution + preference/quiet-hours filtering is **PENDING (HRD-154 / HRD-156)**. So "registering the main group as a subscriber" today == pinning `HERALD_TGRAM_CHAT_ID` in `.env` (see §4).

---

§2. Prerequisites

On the nezha host, in the Herald checkout (the `tools/herald` submodule of the ATMOSphere superproject, per HRD-157):

1. **A built** `pherald` **binary.** Either:
   - place it at `<herald-home>/bin/pherald`, **or**
   - have it on `$PATH`, **or**
   - pass `--build` to `install.sh` (needs a Go toolchain + the Go workspace — `go.work` listing all Herald modules + the `submodules/` checked out).
2. **A populated** `.env` at `<herald-home>/.env` (git-ignored). Copy `.env.example` and set at minimum:
   - `HERALD_CHANNELS=tgram`
   - `HERALD_TGRAM_BOT_TOKEN=<bot token from @BotFather>`
   - `HERALD_TGRAM_CHAT_ID=<numeric id of the ATMOSphere main group>`

    - `HERALD_PROJECT_NAME=ATMOSphere` (pins the Claude Code session-envelope name)
    - For inbound operator commands (`Done:`/`Reopen:`): `HERALD_OPERATOR_IDS=<comma-separated chat ids>`
    - `HERALD_CLAUDE_BIN` if `claude` is not on `$PATH`.
    - Resolution order: a value already exported in the shell wins; `.env` is the fallback (never overrides shell exports).
3. `docs/Issues.md` **present.** `pherald listen` **refuses to boot** without it (§107 fail-loud). It ships in the repo, so this holds in a normal checkout.
4. **The workable-items SSoT** at `docs/workable_items.db`. If absent, `pherald watch` opens an empty schema-only DB and has nothing to diff until HRD-150/HRD-155 materializes the populated DB; `install.sh` warns but does not block on this.
5. **(Linux) systemd + the ability to** `loginctl enable-linger` so the services survive logout/run at boot. `install.sh` attempts `enable-linger` best-effort.

`install.sh` validates all the **hard** prerequisites and exits non-zero (fail-loud) if any is missing — a missing bot token, a missing `docs/Issues.md`, or an unreadable `.env` aborts the install rather than producing a daemon that boots into a silent-failure loop.

---

§3. One-time MTProto login (QA harness only — optional for the daemons)

> The watch/listen **daemons do NOT need MTProto.** They authenticate to Telegram with the **bot** token. MTProto is the **user-account** credential used only by `qaherald` to drive the live round-trip QA tests (HRD-156). You can deploy and run the daemons without ever doing this step; `install.sh` only **warns** if the session is absent.

If you also want the live QA harness (recommended before claiming HRD-156 live evidence):

1. Populate the MTProto vars in `.env` (`HERALD_MTPROTO_APP_ID`, `HERALD_MTPROTO_APP_HASH`, `HERALD_MTPROTO_PHONE`, optionally `HERALD_MTPROTO_PASSWORD`). Get `app_id`/`app_hash` from https://my.telegram.org/apps. See `docs/guides/OPERATOR_CREDENTIALS.md` for the step-by-step, including the Telegram-account-safety warnings (use a real mobile number, not VoIP; set the `recover@telegram.org` email first).

2. Run the one-time interactive login (this is the single §11.4.98-permitted manual bootstrap, done **outside** test execution):

   ```
   ./bin/pherald >/dev/null 2>&1 || true   # ensure built; or use qaherald directly:
   qaherald mtproto login                  # prompts for the SMS/app code (+ 2FA password if enabled)
   qaherald mtproto whoami                 # verifies the persisted session is alive
   ```

   The session persists to `HERALD_MTPROTO_SESSION_FILE` (default `~/.config/herald/mtproto.session`); subsequent runs reuse it with **no** human action.

---

§4. Seed the notification recipients

Because of the routing model in §1, seeding is primarily an `.env` check today:

```
deploy/atmosphere-herald/seed-subscribers.sh            #
        verify .env routes to the main group
```

This confirms `HERALD_TGRAM_CHAT_ID` is set (it is the live routing target) and exits non-zero if it is empty.

Forward-compat (optional, no-op for `watch` today): to pre-seed the Postgres `subscribers` + `subscriber_aliases` tables for the future HRD-156 path:

```
export DATABASE_URL='postgres://…'                      # the
        Herald Postgres
deploy/atmosphere-herald/seed-subscribers.sh --pg
        # upserts subscriber + main-group alias
deploy/atmosphere-herald/seed-subscribers.sh --print-sql  # just
        print the SQL, touch nothing
```

The SQL upserts one `subscribers` row (handle `atmosphere-main`, role `operator`) and a `subscriber_aliases` row mapping `tgram` + the main-group chat id to it, under the RLS tenant (`HERALD_TENANT_ID`, default the all-zero UUID for a single-tenant host). It is idempotent (`ON CONFLICT … DO UPDATE`). This is **not consumed by `pherald watch` yet** — it only pre-populates the table for when HRD-156 lands subscriber resolution.

---

§5. Install

From the Herald checkout root:

```
deploy/atmosphere-herald/install.sh                     #
        autodetect everything, enable + start
```

Useful flags (flags > env > defaults):

Flag	Default	Meaning
`--herald-home DIR`	the repo root above `deploy/`	Herald checkout root
`--env-file FILE`	`<home>/.env`	credentials file
`--pherald-bin PATH`	`<home>/bin/pherald`, else `$PATH`	the binary
`--workable-db PATH`	`<home>/docs/workable_items.db`	the SSoT
`--docs-dir DIR`	`<home>/docs`	trackers root for `listen`
`--build`	off	build `pherald` with the Go toolchain first
`--no-start`	off	install + enable but do not start
`--dry-run`	off	print every action, change nothing

What it does: resolves paths → validates prerequisites (fail-loud) → renders the platform unit templates into the user unit dir → `daemon-reload` → `enable` → `restart`. On Linux it also attempts `loginctl enable-linger` so the daemons survive logout and start at boot.

Re-running is safe and idempotent — it overwrites the rendered units, reloads, and restarts.

---

§6. Verify the daemons are live

This is the **anti-bluff proof step** — "installed" is not "working". Capture the output under `docs/qa/HRD-157-<TS>/` as the §107.x evidence.

6.1 The daemons are up

Linux (systemd):

```
systemctl --user status atmosphere-herald-watch atmosphere-
        herald-listen
journalctl --user -u atmosphere-herald-watch -u atmosphere-
        herald-listen -n 50 --no-pager
```

Expect `active (running)` for both, and in the journal:

- watch: `pherald watch: watching …; poll=1s`
- listen: `pherald listen: starting Telegram getUpdates long-poll loop` + `workable-item mutation enabled (...)`

macOS (launchd):

```
launchctl print gui/$(id -u)/digital.vasic.herald.watch | grep
        -E 'state|pid'
tail -n 50 qa-results/atmosphere-herald-listen.*.log
```

6.2 Outbound proof — a real SSoT mutation reaches Telegram

With `watch` running, mutate one item in the SSoT and confirm a message lands in the main group:

```
# Either via the workable-items tool, or a direct SQLite status
        change:
sqlite3 docs/workable_items.db \
  "UPDATE items SET status='Ready for testing',
        last_modified=datetime('now') \
   WHERE atm_id='ATM-238' AND current_location='Issues';"
```

Within ~1–2s a message like 🔄 `ATM-238 status: In progress → Ready for testing` should arrive in the ATMOSphere main group. Screenshot it + copy the journal line into `docs/qa/HRD-157-<TS>/`.

> If nothing arrives: confirm `--poll` > 0 (default 1s) and that the row actually changed in `items` (the watcher diffs `Repo.List` snapshots; a tracker-only edit may not change the DB rows until the regenerator HRD-150 lands). See §7.

6.3 Inbound proof — a real operator message mutates a row

In the main group, send a message that drives an `item.update` (Claude Code returns the `<<<HERALD-REPLY>>>` block). The dispatcher applies it and replies. Confirm the row changed:

```
sqlite3 docs/workable_items.db \
  "SELECT atm_id, status FROM items WHERE atm_id='ATM-238';"
```

The bidirectional transcript is auto-journaled to the listen daemon's `--qa-out-dir` (`docs/qa/atmosphere-listen-<TS>/transcript.jsonl` + `attachments/`). Commit that dir as the §107.x evidence.

---

§7. Troubleshooting

Symptom	Likely cause	Fix
`install.sh` aborts: `HERALD_TGRAM_BOT_TOKEN empty …`	token not set but `tgram` enabled	Set `HERALD_TGRAM_BOT_TOKEN` in `.env` (fail-loud is intentional).
`install.sh` aborts: `docs/Issues.md not found`	wrong `--herald-home` / `--docs-dir`	Point at the repo root; `pherald listen` requires `Issues.md`.
`install.sh` aborts: `pherald binary not found`	no binary + no toolchain	Pass `--pherald-bin`, or `--build` with the Go workspace set up.
watch: `no channels enabled (HERALD_CHANNELS resolved empty)`	no channels configured	Set `HERALD_CHANNELS=tgram` + the per-channel env.
Mutations don't notify	WAL writes not seen / `--poll 0`	Keep `--poll` > 0 (default 1s); the unit pins `--poll 1s`.
Notify fires but diff empty/wrong	snapshot vs. parser mismatch	Confirm the rows changed in `items`, not just the MD tracker (HRD-150 regenerator pending).
listen: `action=item.update but no ItemMutator configured`	`--db` resolved empty	The unit pins `--db <WORKABLE_DB>`; confirm the path resolved (re-run install).
`CONFIRM …: no pending action for token`	token unknown / already consumed	Re-run the investigation for a fresh token; CONFIRM consumes once.
systemd: `Failed to connect to user scope bus` over SSH	no user D-Bus session	`loginctl enable-linger $(id -un)` (install does this best-effort), then reconnect.
Service flaps / `start-limit-hit`	hard-failing config (e.g. revoked token)	Fix the config, then `systemctl --user reset-failed atmosphere-herald-watch` and restart.
Live Telegram QA "fails"/skips		

Symptom	Likely cause	Fix
	MTProto session not bootstrapped	Expected — see §3; the daemons themselves don't need MTProto.

Logs: `journalctl --user -u atmosphere-herald-watch -f` (Linux) or `tail -f qa-results/atmosphere-herald-*.log` (macOS).

---

§8. Uninstall

```
deploy/atmosphere-herald/uninstall.sh                  # stop +
        disable + remove the units
deploy/atmosphere-herald/uninstall.sh --dry-run        # preview
```

It stops, disables, and removes the units/agents. It does **NOT** delete the workable DB, `.env`, the `pherald` binary, or any `docs/qa/` transcripts (captured evidence is preserved per §107.x). Lingering is left enabled; disable it manually with `loginctl disable-linger $(id -un)` if you want the user manager to stop at logout.

---

§9. References

- `deploy/atmosphere-herald/` — the install tooling + unit templates this runbook drives.
- `docs/guides/WORKABLE_ITEMS_INTEGRATION.md` — the integration architecture, §4 (`pherald watch`), §5 (inbound actions), §4.4 (the recipient routing caveat).
- `docs/guides/OPERATOR_CREDENTIALS.md` — credential setup for every channel + the MTProto bootstrap.
- `pherald/cmd/pherald/watch.go`, `pherald/cmd/pherald/listen.go` — the daemon command sources (authoritative for flags + env).
- `docs/Issues.md` — HRD-152 / HRD-153 / HRD-156 / HRD-157 work-items.

Sources verified

Verified 2026-05-30:

- Telegram Bot API — `getUpdates` long-poll semantics + webhook mutual-exclusivity: https://core.telegram.org/bots/api#getupdates (fetched 2026-05-30; confirms `getUpdates` is the long-polling receive method and will not work while an outgoing webhook is set). The watch/listen daemons use `getUpdates` (no webhook), consistent with this.
- systemd `loginctl enable-linger` — user services persist after logout + start at boot: https://manpages.debian.org/bookworm/systemd/loginctl.1.en.html (fetched 2026-05-30; quote: "a user manager is spawned for the user at boot and kept around after logouts … allows users who are not logged in to run long-running services").
- **Negative finding (documented per §11.4.99(B)):** the canonical systemd man pages at `freedesktop.org/software/systemd/man/` and the Arch Wiki `Systemd/User` page were bot-blocked (HTTP 403 / Anubis challenge) at fetch time 2026-05-30 and

could not be machine-verified. The `systemctl --user enable/start/status/restart`, `daemon-reload`, `~/.config/systemd/user` unit path, and `journalctl --user -u` usages in `install.sh` and this runbook rely on long-stable systemd user-instance semantics (corroborated by the `loginctl` manpage above); re-verify against the official systemd man pages before the next vN.0.0 release boundary or on any systemd breaking-change announcement (§11.4.99(C) cadence).

- Telegram MTProto `app_id`/`app_hash` provisioning + account-safety guidance: cross-referenced in `docs/guides/OPERATOR_CREDENTIALS.md` (which carries its own `Sources verified` footer against https://core.telegram.org/api/obtaining_api_id); not re-fetched here to avoid duplicating that doc's verification.